

21/10/25

I began the project by studying the ITCH protocol and implementing a simulated ITCH message generator (IMG) over UDP multicast, sending 'Add Order' messages to a multicast IP and successfully receiving them on a second device via IGMP membership.

24/10/25

I optimized the ITCH message serialization by replacing virtual functions with static polymorphism using templates, reducing memory overhead and CPU branch hazards, and implemented serialized message sending to minimize network payload size.

26/10/25

I added random ITCH message generation using function pointers, observed network inefficiencies due to small packet payloads, and noted the need for batching messages before sending to improve throughput and reduce congestion.

29/10/25

I developed the Market Data Feed Handler (MDFH) client to receive multicast UDP traffic, learned to properly bind sockets to INADDR_ANY for multicast, and ensured kernel-level packet delivery via IP_ADD_MEMBERSHIP.

01/11/25

I implemented parsing of received ITCH messages in MDFH with cache-aligned buffers, branchless timestamp decoding, and a CRTP-based static polymorphism logging system, achieving efficient, low-latency parsing while preparing for future batched reads and latency measurement.

04/11/25

I implemented batching in the ITCH message server to maximize UDP payload utilization, reduce CPU overhead, and observed a 17× performance improvement compared to sending individual messages.

07/11/25

Sequence numbers were added to messages and the client updated to parse batched messages, detect lost packets, and benchmark the retry buffer, confirming correct parsing and handling of message order.

08/11/25

I developed a sequence-number-based lost message detection system with sliding windows and gap timers, handling duplicates and out-of-order packets, and observed significant packet loss over Wi-Fi multicast, highlighting the need for testing on switched Ethernet.

11/11/25:

Migrated MDFH to Linux and set up a switched Ethernet network, bypassing kernel socket processing with raw AF_PACKET sockets to directly handle L2 frames, filter by multicast IP and port, and pass payloads to `parseMessage()`. Benchmarking showed a 507x throughput improvement over WiFi, and implementing TPACKETv1 further increased throughput by 12% by using a shared mmap'd ring buffer.

14/11/25:

Implemented TPACKETv3 to process packets block-by-block, reducing context switches and

improving memory efficiency, but throughput plateaued around 1.4–1.5M messages/sec due to CPU limits in single-threaded parsing. Profiling revealed that the `gapTimer()` sequencer logic consumed nearly half the CPU, highlighting the need to switch to condition variable notifications and enable multithreading for further optimization.

17/11/25:

Focused on making the sequencer fully lock-free by converting the bitset to an atomic array and using CAS/fetch_add for concurrent updates, while optimizing the `gapTimer` with CPU pinning and real-time scheduling (SCHED_FIFO) to minimize migrations, cache misses, and overhead, achieving stable, low-latency performance.

20/11/25:

Designed and implemented a multithreaded MDFH using a single network thread and multiple parser threads with lock-free SPSC queues, carefully managing buffer ownership and CPU pinning to avoid cache line bouncing, ultimately hitting a hardware-limited throughput ceiling despite software optimizations (running on a 4 logical core, 2 physical core system with a single NUMA node spanning all of memory).